

Transitix Companion — Sprint 2

Branch **RAI-spedition-driver-app** · profil **documents**

Versiune companion **v1.1.0** · build **8** · deschis **28 august 2026**

Perioada: de la pull *driver-app* (după Sprint 1) până la închiderea sprintului (anunțată de PO)

Companion izolat

Paddle OCR

VM + Cloudflare

App șofer

Changelog de testare pentru **RAI Spedition — app documente** (nu Transitix TMS complet). Sprint 1 = upload șofer + OCR Paddle (fișierul `ChangeLog-RAI-spedition-driver-app.html`).

CHANGELOG

App companion izolat (profil documents)

- **Profil documents** activat din `vite --mode companion` — nu mai depinde de un fișier `.env.companion` uitat; build fără mod companion = Transitix full.
- **Birou:** meniu redus — doar **Avize / Rapoarte** și **Rapoarte**; landing `/avize` ; orice rută necunoscută redirectionează la `/avize` .
- **Șofer:** shell dedicat `DriverAppDocuments` (upload + profil), separat de `DriverApp.jsx` (app șofer Transitix full cu curse/CMR).
- **Login companion:** subtitlu „Autentifică-te în RAI Spedition — Documente”; link **Creează cont** ascuns (companion = conturi invitate).
- **Navigare post-login:** birou → `/avize` ; șofer → `/driver-app` ; `returnTo` respectat doar pe rutele companion (`/avize` , `/reports`).
- **Layout birou:** fără tur ghidat și fără căutare globală — sidebar minimal; clopoțelul de notificări rămâne în topbar.
- **Docker companion:** `docker-compose.companion.yml` — Postgres `:5435` (DB `transitix_rai`), Paddle `:8101` (mapat din 8100 container).
- **Env:** `companion.env.example` (root) + `server/companion.env.example` — API `:3002` , `JWT_SECRET` și DB proprii, `PADDLE_OCR_URL=http://127.0.0.1:8101` .
- **Dev:** `npm run dev:companion` → Vite `0.0.0.0:5174` ; API pe `:3002` .
- **Producție VM:** `npm run start:companion` — `vite build --mode companion` + API servește `dist/` (un singur origin, fără HMR prin tunnel).
- **Backup:** `npm run db:backup -- --keep 14` — `pg_dump` + arhivă folder uploads; ambele părți sunt necesare pentru refacerea unui export.
- **README:** secțiune dedicată „RAI Spedition companion” cu pași Docker, migrate, seed, start.

Branding și versiune companion

- Titlu produs: **RAI Spedition — Documente** (`package.json` → `companion.title` , suprascriibil prin `VITE_APP_TITLE`).
- Tagline sidebar: **Companionul tău pentru documente** (`VITE_APP_TAGLINE` opțional).
- Versiune afișată în **login** (în card), **sidebar/topbar** birou și **titlul tab-ului** browser: **v1.1.0 - build 8**.
- Număr companion în `package.json` → `companion` (`version` + `build`), decuplat de Transitix 1.12.x; hook-ul semver nu atinge blocul `companion` .
- Script `scripts/resolve-app-build.js` — sursă unică pentru `__APP_VERSION__` , `__APP_BUILD__` , titlu și profil la compile; build numeric (nu hash git).
- Evoluție în sprint: v1.0.0 → build 4 → **v1.1.0** build 5 → 6 → 7 → **8** (ultimul = layout topbar/sidebar).
- Profilul de build (`documents` / `full`) în `resolveAppProfile(mode)` , testat în `resolve-app-build.test.js` — regresia care livra tăcut Transitix full în loc de companion era nedetectabilă din config.

OCR — Paddle unic (Google Vision eliminat)

- **Un singur extractor** — `extractBatchDocuments` în `documents.js` , peste `readText.js` ; fișierele `avizVision.js` și testele Vision șterse.
- Ordine citire: **PDF cu text layer** (pdf-parse, gratuit) → **PaddleOCR** local; fără fallback la alt provider.
- Același flux pentru upload șofer, loturi birou și butonul **Re-extrage** pe `/avize` .
- **CMR / integrations:** endpoint-ul `/api/integrations/llm` nu mai apelează Vision; pozele CMR trec prin coada avize, nu prin stub separat.
- `GET /api/health` — câmp `capabilities.ocr` : `paddle` / `paddle-down` / `false` (probe la sidecar, TTL 15s); `vision` scos din health.
- Banner galben pe `/avize` când `OCR = paddle-down` : upload merge, câmpurile rămân goale până repornești Paddle și apeși Re-extrage.
- După upload șofer: OCR doar pe **documentele noi** (`documentIds`); lock per batch; erori per-document nu blochează restul lotului.
- **Batch doar înainte:** extracția nu rescrie rânduri deja `confirmed` ; status batch nu revine din `confirmed` în `extracted` .
- **Timeout ≠ eșec:** la `OCR_TIMEOUT` documentul rămâne `uploaded` în coadă (nu se marchează „gol”); operatorul poate reîncerca.

OCR — sidecar Paddle îmbunătățit

- **PDF scanat rasterizat în sidecar** (PyMuPDF) — Node nu mai are nevoie de toolchain imagini; sidecar-ul primește PDF-ul întreg.
- Documente multi-pagină: OCR pe fiecare pagină; răspuns cu `pages` , `total_pages` , `truncated` când depășește plafonul (`PADDLE_OCR_PDF_PAGES` , default 40).
- **Rotație reutilizată:** unghiul câștigător de pe pagina 1 se încearcă primul pe paginile următoare (scanner alimentează la fel).
- Auto-rotație 0/90/270/180 cu early-exit când TPO/PSL/plăcuță sunt deja lizibile pe orientarea curentă.
- Flag `pages_truncated` stocat pe rândul avizului când citirea a fost parțială — vizibil în evenimente document.

OCR — bugete timp (interactiv vs background)

- **Interactiv** (cineva așteaptă): timeout 45s (`OCR_INTERACTIVE_TIMEOUT_MS`); max 3 pagini (`OCR_INTERACTIVE_MAX_PAGES`).
- **Background** (upload șofer, lot mare): timeout 300s+ (`OCR_TIMEOUT_MS`), crește cu numărul de pagini.
- Document > 3 pagini la **Re-extrage** pe `/avize` → răspuns **202** cu `extraction_pending: true` ; OCR continuă în fundal.

App șofer — shell și UX upload

- **DriverAppDocuments**: layout mobil (safe-area, `100dvh`), lățimi responsive iPhone SE → iPad/Surface; tab-uri Upload + Profil.
- **Avertisement blur** înainte de trimitere (heuristică Laplacian în `imageQuality.js`) — refotografiază sau trimite oricum.
- **Throttle client** (`uploadThrottle.js` , 20/min) înainte de limita serverului (30/min) — mesaj cu secunde rămase.
- **Polling** listă „Trimise recent” la 8s cât timp status = Încărcat — doar când tab-ul e vizibil (nu din buzunar).
- Verificare **număr fișiere** (max din API) și **15 MB** per fișier înainte de upload.
- Erori upload server traduse (`uploadErrorMessage`): „Prea multe fișiere odată...”, nu mesajul raw multer.

Mesaje eroare în română (client + API)

- **API auth**: login, refresh, me — mesaje operator („Email sau parolă greșite”, „Sesiunea a expirat” etc.).
- **Middleware**: 401/403 birou și admin — texte cu pas următor, nu „Invalid or expired token”.
- **Client notify.js** : detectează offline („Fără conexiune”); titluri după status HTTP; preferă mesajul serverului față de boilerplate englezesc.
- **Login**: erori prin `friendlyErrorMessage(err)` , nu `err.message` brut.
- Upload-uri, confirmări client, company — mesaje API traduse acolo unde operatorul le citește.

Deploy VM, tunnel și documentație changelog

- **Dezvoltare** — 3 terminale: API (`:3002`), UI companion (`:5174`), `cloudflared tunnel --url http://127.0.0.1:5174` . Pentru punerea la dispoziția clientului, vezi „Publicare” mai jos: build servit de API, pe un tunel cu nume.
- Quick tunnel Cloudflare (`*.trycloudflare.com`) — `allowedHosts` în Vite companion. Rămâne valabil doar pentru dezvoltare.
- Proxy API companion forțează port **3002** (evită proxy implicit spre Transitix full pe 3001).
- **Seed**: conturi demo `admin@transitix.ro` / `sofer@transitix.ro` ; parole reîmprospătate la fiecare `npm run db:seed` .
- Changelog RAI: Sprint 1 închis; Sprint 2 (acest fișier) deschis; șterse changelogs Transitix TMS din `changelogsSprints/` ; whitelist HTML/PDF în `.gitignore` .

Publicare — tunel cu nume, proxy și headere (build 6)

- **TLS rămâne la Cloudflare**, nu în aplicație: certificatul de la marginea tunelului bate un certificat auto-semnat pe VM, care ar fi un pas înapoi plus reînnoiri de întreținut.
- **Tunel cu nume în locul celui rapid**. Un quick tunnel are certificat real, dar adresa `*.trycloudflare.com` se schimbă la fiecare repornire, deci linkul salvat de client moare după un reboot. Șablon și pași: `docker/cloudflared/config.example.yml` (hostname permanent, `connectTimeout` ridicat pentru OCR lung, notă că SSE nu are voie tamponat, port `3002`).
- `TRUST_PROXY` — **oprit implicit**. A crede `X-Forwarded-*` fără nimic în față permite oricui să pretindă orice adresă. Acceptă și număr de hopuri.
- `server/src/lib/security.js` : `nosniff` , `X-Frame-Options: DENY` , `frame-ancestors 'none'` , `Referrer-Policy` .
- **HSTS doar pe HTTPS**, decis per cerere din `req.secure` — un dezvoltator pe localhost nu-și fixează browserul pe o schemă pe care serverul de dezvoltare n-o vorbește.
- Fără CSP complet, intenționat: aplicația încarcă bundle propriu, stiluri inline și previzualizări pe blob-URL; o politică netestată ar strica ecranul în tăcere.
- `CLIENT_ORIGIN` mutat pe hostname-ul `https://` , aceeași origine ca API-ul.

Întreținere — curățare uploads (build 6)

- `npm run uploads:prune` — șterge **doar orfani**: fișiere pe care nicio linie din bază nu le referențiază.
- **Un fișier referențiat nu se șterge niciodată**, oricât de vechi. `aviz_export_log.snapshot` reproduce cifrele de pe anexă, dar numai fișierul arată clientului de unde vin. Retenția prin vechime e o decizie contabilă cu termen legal, nu una de spațiu pe disc — deliberat neimplementată.
- Trei paznici: **dry-run implicit** (`--delete` cere intenție), `--min-age-days` (7) ca un upload care își așteaptă rândul să nu dispară, și **oprire dacă schema s-a schimbat** — o coloană redenumită ar face fișierele ei să pară orfane.
- Se oprește și dacă baza nu conține nicio companie, dar există fișiere pe disc — semn de `DATABASE_URL` greșit.
- O coloană nouă care ține o cale de upload **trebuie adăugată în** `FILE_COLUMNS` , altfel fișierele ei devin ștergibile.

Calibrare prag claritate (build 6)

- `/dev/sharpness` — pagină **doar în build de dezvoltare**: adaugi avize fotografiate, le marchezi Citibil / Neclar, primești pragul sugerat lângă cel din cod și câte greșeli face fiecare.
- Rulează **exact codul din producție** (același decodor, aceeași reducere la 640px, același Laplacian). Un script Node cu alt decodor ar scala diferit și ar da un prag care se poartă altfel în browser.
- `suggestThreshold()` nu tratează egal cele două greșeli: un avertisment fals costă o apăsare (verificarea nu blochează), o poză mișcată ratată costă un document ilizibil. Ratările cântăresc dublu, la egalitate pragul urcă.
- Spune când grupele se suprapun și când sunt prea puține exemple ca sugestia să fie credibilă.
- Verificat absent din build-ul de producție — nici ruta, nici `suggestThreshold` (scos de tree-shaking).
- Verificarea de claritate rulează acum și pe upload-ul din birou, dar **fără să întrerupă** un lot de scanări: numește fișierul slab și continuă.

Sidecar — verificare fără motorul ML (build 7)

- `tools/paddle-ocr/check_logic.py` rulează tot ce e în jurul recunoașterii, cu PaddleOCR înlocuit de un dublu: rasterizare PDF, plafon de pagini, raportarea trunchierii, căutarea orientării, contractul endpoint-ului.
- Motivul: motorul are sute de megaocteți și descarcă modele, deci pe o mașină fără el logica din jur rămânea nerulată. **12 verificări**, ies cu cod diferit de zero la eșec.
- Confirmă economia din re folosirea orientării: `passes per page: [4, 1, 1]` — prima pagină caută toate cele patru unghiuri, următoarele costă câte o trecere.
- Copiat în imagine, deci rulabil și în container: `docker compose -f docker-compose.companion.yml exec paddle-ocr python /app/check_logic.py`.
- Verificat separat că toate pinurile (`pymupdf` , `paddlepaddle` , `paddleocr` , `numpy` , `pillow`) au wheel pentru cp310/manylinux — build-ul nu compilează din sursă.
- `.gitignore` : `__pycache__/` pentru scripturile Python din tooling OCR.

Rezumat build-uri Sprint 2

- **5** — profil companion, Paddle unic, UX șofer, mesaje RO (`notify.js`), changelog Sprint 2 inițial.
- **6** — securitate proxy/HSTS, `uploads:prune` , blur birou, calibrare `/dev/sharpness` , tunel Cloudflare cu nume, `notifyError` cu un singur argument.
- **7** — `check_logic.py` (12 verificări sidecar fără ML).
- **8** — topbar: profil + notificări în colțul dreapta; sidebar: spațiu deasupra logo-ului.

BUGFIX

Login 401 pe VM

- **Seed:** parole demo reîmprospătate la re-run (`admin123` / `sofer123`) — conturile existente nu rămâneau cu hash vechi.
- **Proxy Vite:** browser pe `:3001` , curl pe `:3002` — mod companion forțează proxy API spre **3002**.
- **Login UI:** erori afișate prin `friendlyErrorMessage` (respectă status HTTP 401 vs offline).

App companion nu se încărca (ecran alb)

- Import `ProtectedRoute` restaurat în `App.jsx` (lipsea după refactor routing).
- `OfficeRoutes` randat ca fragment `{OfficeRoutes()}` — react-router v6 nu acceptă componentă ca copil direct.

Cloudflare tunnel blocat de Vite

- Host `*.trycloudflare.com` respins — adăugat în `allowedHosts` când `mode === companion` .
- Dev server companion ascultă pe `0.0.0.0:5174` (nu doar localhost).

OCR rămânea pe „Încărcat”

- Batch deschis: OCR reprocessa toate pozele vechi din lot înainte de cea nouă.
- Timeout 2 min tăia Paddle pe VM cu CPU limitat (rotații multi-unghi).
- Remediat: OCR doar fișiere noi (`documentIds`), timeout background 5 min, erori per-document nu opresc batch-ul.
- Lock `runBatchExtract` — două extracții simultane pe același batch nu se mai calcă.

Re-extrage pe /avize golea câmpurile

- Butonul folosea calea Vision/stub când lipsea cheia Google — înlocuit cu același Paddle ca upload-ul șoferului.
- Eliminat al doilea extractor (`avizOcr.extractAvizFromFile` pe Vision) care suprascrisa TPO și câmpurile cu stub.

Extracție anula confirmarea operatorului

- Background OCR putea rescrie status batch/document după ce operatorul confirmase pe `/avize`.
- Remediat: batch nu revine din `confirmed`; update status document doar când rândul e încă `uploaded`.

Timeout OCR marca documentul ca eșuat

- La expirarea ceasului, documentul era marcat failed și dispărea din coada de re-extrage.
- Remediat: cod `OCR_TIMEOUT` — rândul rămâne `uploaded`, mesaj cu timpul, reîncercare posibilă.

Versiune companion confuză (hash git)

- Build afișat ca hash git scurt — înlocuit cu număr `companion.build` din `package.json` (build 4 → 5).

Mesaje eroare — traducător și status HTTP (build 5)

- `notify.js` rescris: detectare offline, titluri după status (401, 403, 429), preferă mesajul serverului față de boilerplate englezesc.
- Login, înregistrare, resetare parolă: `friendlyErrorMessage(err)` în loc de `err.message` brut.
- `Vehicles.jsx`: toast-uri directe înlocuite cu `notifyError('Text', err)` ca statusul HTTP să ajungă la traducător.

Toast cu un singur argument (build 6)

- Apeluri de forma `notifyError('Mesaj precis')` primeau și o descriere generică „A apărut o eroare neașteptată” — inutilă și confuză.
- Remediat: un argument = doar titlul; al doilea argument opțional activează traducătorul și titlul după status.

Topbar birou — iconițe centrate (build 8)

- Profil și notificări apăreau la mijlocul header-ului în profilul companion (fără căutare globală).
- Cauză: grid cu trei coloane — al doilea copil ajungea în coloana din mijloc când `GlobalSearch` lipsea.
- Remediat: layout flex; grupul din dreapta cu `m1-auto`, lipit de colțul dreapta.

Logo sidebar lipit de marginea browserului (build 8)

- Zona de brand din sidebar nu avea spațiu deasupra logo-ului.
- Adăugat padding sus (`safe-area-inset-top` pe telefon) ca logo-ul să nu fie lipit de headerul browserului.

Testare și acoperire

- Teste unitare noi: `imageQuality.test.js` (inclusiv `suggestThreshold`), `uploadThrottle.test.js`, `appProfile.test.js`, `readText.test.js` (capability OCR, bugete timp), `notify.test.js` (status, offline, boilerplate), `resolve-app-build.test.js` (profil din mode), `security.test.js` (proxy, HSTS), `pruneUploads.test.js` (decizia de ștergere, izolată de disc și de bază).
- **Total la build 8: 1436+ teste unitare (94 fișiere)** și 201 teste API (12 fișiere), lint curat.
- Logica sidecar-ului: `python tools/paddle-ocr/check_logic.py` — 12 verificări, fără PaddleOCR.
- Teste API rute: `driverDocuments.api.test.js` (upload + extracție), `avize.api.test.js` (extract, health OCR) — necesită `TEST_DATABASE_URL` cu sufix `_test`.
- Verificare manuală VM: health → upload șofer → status trece din Încărcat → admin /avize cu sursă `paddle` și badge De la șofer.

Testare rapidă (VM companion)

Variantă dev — 3 terminale (setup curent pe VM):

1. `docker compose -f docker-compose.companion.yml up -d --build`
2. Terminal 1: `npm run dev --prefix server (API :3002)`
3. Terminal 2: `npm run dev:companion (UI :5174 , proxy spre API)`
4. Terminal 3: `cloudflared tunnel --url http://127.0.0.1:5174`
5. `server/.env` : `CLIENT_ORIGIN = URL-ul Cloudflare, TRUST_PROXY=1 , PADDLE_OCR_URL=http://127.0.0.1:8101`
6. Verifică versiune în login/topbar: **v1.1.0 - build 8**

Variantă producție — un singur port (tunnel spre API, UI servit din build):

1. `docker compose -f docker-compose.companion.yml up -d --build` — Postgres :5435 , sidecar OCR :8101 .
2. `curl http://127.0.0.1:8101/health` — `engine_loaded:false` e normal la pornire.
3. `docker compose -f docker-compose.companion.yml exec paddle-ocr python /app/check_logic.py` — caută passes per page: [4, 1, 1] .
4. `cp server/companion.env.example server/.env` → JWT + `CLIENT_ORIGIN` · `npm run db:migrate`
5. `npm run start:companion` — build + API pe :3002 ; tunnel spre `http://127.0.0.1:3002` .
6. `curl -s http://127.0.0.1:3002/api/health` → `capabilities.ocr = paddle` .
7. **Proba OCR:** aviz multi-pagină pe /avize ; peste 3 pagini → extracție în fundal (202).
8. Calibrare blur (dev): /dev/sharpness cu avize reale fotografiate.